

Rapport technique - Module Auth ERP SaaS

Analyse complete du modele, travail realise etape par etape, risques restants et prochaines ameliorations recommandees.

Champ	Valeur
Projet	erp-intelligent-saas / apps/api
Stack	Node.js, Fastify, TypeScript, Prisma, PostgreSQL, npm
Module	Authentification, sessions, JWT, refresh tokens, RBAC, email verification, reset password
Date	2026-06-04 17:09

Resume executif

Le module auth est maintenant sur une base solide: routes Fastify structurees, validation Zod, JWT courts, refresh tokens opaques hashes, sessions DB, CSRF, rate limiting, outbox email, audit logs, RBAC store/organization et CI PostgreSQL. Les prochains gains majeurs concernent le durcissement cryptographique, la concurrence refresh, l'isolation multi-tenant et l'industrialisation production.

1. Architecture actuelle du module

Le module suit une architecture en couches. Les routes exposent l'API HTTP, les controleurs valident les payloads et formatent les reponses, le service contient la logique metier, les middlewares appliquent l'authentification et l'autorisation, les plugins gerent les concerns transverses, et Prisma represente le modele persistant.

Couche	Fichiers	Role
Routes	auth.route.ts	Declaration endpoints, CSRF, rate limit, middlewares.
Controleurs	auth.controller.ts	Validation HTTP, appels services, cookies et reponses uniformes.
Services	auth.service.ts	Register, login, refresh, logout, verification email, reset, change password.
Middlewares	auth.middleware.ts, role.middleware.ts	Recharge user/session/roles depuis DB et applique RBAC.
Plugins	jwt.ts, security.ts, errorHandler.ts	JWT, cookies, CORS, CSRF, rate limit, erreurs centralisees.
Prisma	schema.prisma, migrations	Users, sessions, refresh tokens, auth tokens, audit logs, outbox, RBAC.
Jobs	cleanupTokens.ts, processEmailOutbox.ts	Maintenance des tokens et traitement email asynchrone.

2. Travail realise etape par etape

- 1 Audit initial du module auth: routes, controleurs, services, middlewares, Prisma, utilitaires et configuration.
- 2 Identification des risques: migration baseline, SMTP bloquant, exposition tokens dev, CSRF incomplet, service trop volumineux, RBAC peu applique, tests DB legers.
- 3 Durcissement refresh tokens: tokens opaques, hash HMAC, sessions DB, rotation atomique et detection reuse.
- 4 Ajout verification email, forgot-password, reset-password et change-password avec tokens en base.
- 5 Creation d'un EmailOutbox pour separer le flux metier de l'envoi SMTP.
- 6 Ajout CSRF sur les routes sensibles, y compris login et refresh.
- 7 Extension du rate limit sur register, login, forgot-password, reset-password, verify-email et refresh.
- 8 Rechargement du user, role, stores et memberships depuis PostgreSQL dans requireAuth.
- 9 Ajout RBAC global, store-level role, organization/membership et middlewares de permission.
- 10 Ajout audit logs pour evenements sensibles: register, login, refresh, logout, access denied, reset et password change.
- 11 Ajout jobs maintenance: purge tokens expires et worker outbox email.
- 12 Ajout documentation docs/auth.md, seed permissions, workflow CI PostgreSQL, tests unitaires et tests integration conditionnels.

3. Ameliorations integrees

Sujet	Avant	Maintenant
Refresh token	Risque de reutilisation et rotation incomplete.	Rotation atomique, famille revokee en cas de reuse detecte.
Reset/verification token	Consommation separable de l'action metier.	Consommation dans transaction metier avec update conditionnel.
SMTP	Pouvait bloquer register/reset.	Outbox asynchrone, retry, lock PROCESSING, recovery stale lock.
CSRF	Pas applique partout.	Applique aux routes auth state-changing.
Rate limit	Principalement login.	Etendu aux routes auth sensibles.
RBAC	Role global et permissions peu exploitees.	Store roles, memberships organization, guards permission.
Secrets	Risque de config staging/prod.	Validation env stricte et parsing boolean explicite.
Migrations	Baseline potentiellement piegeuse.	Baseline conservee, migration incrementale ajoutee.

4. Securite actuelle

- Access token JWT court avec expiration 15 minutes.
- Refresh token opaque genere aleatoirement et stocke sous forme hash HMAC.
- Cookie refresh HttpOnly, Secure en production, SameSite strict.
- CSRF par cookie secret et header x-csrf-token.
- Validation Zod sur les payloads register/login/reset/change-password.
- Audit logs pour les evenements critiques et refus d'accès.
- Prisma limite les injections SQL via requetes typees et parametrees.
- CORS base sur allowlist multi-origine via CORS_ORIGIN.
- Session et role recharges depuis DB a chaque requireAuth.

5. Points faibles restants

Priorite	Point	Risque	Recommandation
Critique	bcrypt + pepper	bcrypt ignore au-dela de 72 bytes; password + pepper peut tronquer le pepper.	Pre-hasher avec HMAC-SHA256 puis bcrypt, ou migrer vers Argon2id.
Haute	Secret commun	PASSWORD_PEPPER sert aussi aux hashes de tokens; blast radius trop large.	Ajouter TOKEN_HASH_SECRET separe et strategie de rotation.
Haute	Refresh concurrent	Deux refresh simultanes peuvent provoquer faux positif reuse et revoke la famille.	Ajouter grace window/idempotence courte avec rotatedAt et replacedByTokenId.
Haute	Outbox pas transactionnelle	Si EmailOutbox.create echoue apres creation user/token, email non envoye.	Creer l'outbox dans la transaction ou ajouter endpoint resend-verification fiable.
Haute	Rate limit memoire	Faible en multi-instance et derriere proxy mal configure.	Redis store + trustProxy controle par env.
Haute	RBAC tenant	User.role global peut donner trop de pouvoir dans un ERP multi-tenant.	Separer PlatformRole et Membership.role.
Moyenne	Store.organizationId optionnel	Boundary tenant incomplet pour routes store.	Migration data puis organizationId obligatoire, ou guard requireStoreInOrganization.
Moyenne	Tokens dev	EXPOSE_AUTH_TOKENS peut etre active en staging non-production.	Limiter a NODE_ENV=test ou localhost strict.
Moyenne	Validation token	Schema accepte min(64) alors que les tokens generes font 128 hex chars.	Utiliser length(128) + regex hex.
Moyenne	Shadow database	Diff complet migrations -> schema impossible sans shadow DB.	Ajouter SHADOW_DATABASE_URL dans prisma.config.ts et CI.

6. Plan d'amelioration recommande

Phase 1 - Durcissement crypto

- Corriger le hash password: HMAC pre-bcrypt ou Argon2id.
- Ajouter TOKEN_HASH_SECRET separe pour refresh/auth tokens.
- Prevoir compatibilite ancienne version pendant migration.
- Ajouter tests hash/verify et tests reset token.

Phase 2 - Robustesse refresh/session

- Ajouter rotatedAt et replacedByTokenId sur RefreshToken.
- Rendre le refresh concurrent idempotent pendant une fenetre courte.
- Conserver revoke family pour reuse reel hors fenetre.
- Tester double refresh concurrent, reuse vole, session expiree.

Phase 3 - Outbox et emails production

- Creer les emails outbox dans la transaction metier.
- Ajouter resend verification avec rate limit.
- Ajouter metriques sent/failed/attempts/latency.
- Planifier worker email via cron, queue ou process dedie.

Phase 4 - Multi-tenant ERP

- Clarifier PlatformRole vs Membership.role.
- Rendre Store.organizationId obligatoire apres migration data.
- Ajouter guards combines organization + store + permission.
- Completer seed permissions par module ERP.

Phase 5 - Industrialisation

- Redis rate-limit et trustProxy.
- SHADOW_DATABASE_URL pour validation migrations en CI.
- Tests integration DB obligatoires en pipeline.
- Logs structures Pino pour audit, email, erreurs et securite.

7. Architecture cible recommandee

Fichier / Service	Responsabilite cible	Benefice
auth.controller.ts	HTTP, validation, cookies, reponse.	Controleurs fins et previsibles.
auth.service.ts	Orchestration auth generale.	Moins de logique bas niveau.
session.service.ts	Sessions, refresh rotation, reuse detection.	Isolation du flux le plus sensible.
password.service.ts	Hash, verify, reset, change-password.	Migration Argon2id plus simple.
auth-token.service.ts	Creation/consommation tokens verification/reset.	Atomicite centralisee.
email-outbox.service.ts	Queue, claim, retry, recovery.	Decouplage SMTP/auth.
authorization.service.ts	RBAC/ABAC, roles tenant/store.	Permissions ERP coherentes.
repositories/*.ts	Acces Prisma par aggregate.	Tests unitaires plus simples.

8. Criteres d'acceptation production

- npm test, npm run build et npx prisma validate passent.
- npm run test:integration passe avec RUN_DB_TESTS=true et DB PostgreSQL jetable.
- npx prisma migrate deploy passe sur base vide et base deja baselinee.
- Aucun token sensible expose hors test/local explicitement autorise.
- Rate limiting distribue actif en multi-instance.
- Tous les endpoints ERP metier utilisent requireAuth + guard permission/tenant.
- Jobs cleanup/outbox planifies, supervisees et documentees.
- docs/auth.md tient a jour env vars, migrations, procedures et incidents.

9. Checks confirmes pendant l'audit

Commande	Resultat	Observation
npm test	OK	6 tests passent.
npm run build	OK	Compilation TypeScript OK.
npx prisma validate	OK	Schema Prisma valide.
npm run test:integration	OK / skip	2 tests DB presents, skippees sans RUN_DB_TESTS=true.

10. Conclusion

Le module auth est passe d'un modele fonctionnel a une base robuste et bien documentee. Les risques restants sont des sujets de maturite production: cryptographie, concurrence, isolation tenant, outbox transactionnelle, rate limiting distribue et verification migrations. La priorite absolue est de traiter crypto + refresh concurrent, puis d'industrialiser outbox, tenant guards et CI DB.